Rw Motor Torque

Component Design Document

1 Description

Reaction wheel motor torque mapping. Maps the commanded body control torque onto the available reaction wheels along the selected control axes, adds a null space term that steers the wheel speeds toward the desired speeds held as a parameter without disturbing the body, and sends the per-wheel motor torques straight to the wheel interface. Wraps the RwMotorTorqueAlgorithm C++ algorithm via its C shim.

2 Requirements

No requirements have been specified for this component.

3 Design

3.1 At a Glance

Below is a list of useful parameters and statistics that give a quick look into the makeup of the component.

- **Execution** - *passive*
- **Number of Connectors** - 4
- **Number of Invokee Connectors** - 2
- **Number of Invoker Connectors** - 2
- **Number of Generic Connectors** - *None*
- **Number of Generic Types** - *None*
- **Number of Unconstrained Arrayed Connectors** - *None*
- **Number of Commands** - *None*
- **Number of Parameters** - 5
- **Number of Events** - *None*
- **Number of Faults** - *None*
- **Number of Data Products** - *None*
- **Number of Data Dependencies** - 2
- **Number of Packets** - *None*

3.2 Diagram

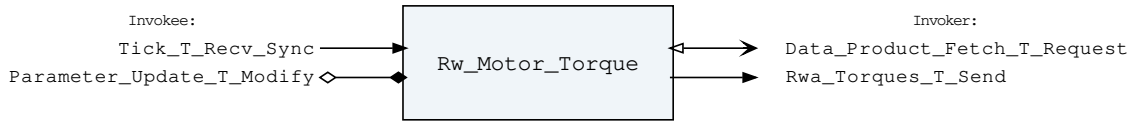


Figure 1: Rw Motor Torque component diagram.

3.3 Connectors

Below are tables listing the component's connectors.

3.3.1 Invokee Connectors

The following is a list of the component's *invokee* connectors:

Table 1: Rw Motor Torque Invokee Connectors

Name	Kind	Type	Return_Type	Count
Tick_T_Recv_Sync	recv_sync	Tick.T	-	1
Parameter_ Update_T_Modify	modify	Parameter_ Update.T	-	1

Connector Descriptions:

- **Tick_T_Recv_Sync** - Run the algorithm up to the current time.
- **Parameter_Update_T_Modify** - The parameter update connector.

3.3.2 Invoker Connectors

The following is a list of the component's *invoker* connectors:

Table 2: Rw Motor Torque Invoker Connectors

Name	Kind	Type	Return_Type	Count
Data_Product_ Fetch_T_Request	request	Data_Product_ Fetch.T	Data_Product_ Return.T	1
Rwa_Torques_T_ Send	send	Rwa_Torques.T	-	1

Connector Descriptions:

- **Data_Product_Fetch_T_Request** - Fetch a data product item from the database.
- **Rwa_Torques_T_Send** - Send the computed per-wheel motor torques directly to the wheel interface component. Only honored by the receiver while the wheels are in closed-loop control.

3.4 Interrupts

This component contains no interrupts.

3.5 Initialization

Below are details on how the component should be initialized in an assembly.

3.5.1 Component Instantiation

This component contains no instantiation parameters in its discriminant.

3.5.2 Component Base Initialization

This component contains no base class initialization, meaning there is no `init_Base` subprogram for this component.

3.5.3 Component Set ID Bases

This component contains commands, events, packets, faults, or data products that require a base identifier to be set at initialization. The `set_Id_Bases` procedure must be called with the following parameters:

Table 3: Rw Motor Torque Set Id Bases Parameters

Name	Type
`Parameter_Id_Base`	`Parameter_Types.Parameter_Id_Base`

Parameter Descriptions:

- **`Parameter_Id_Base`** - The value at which the component's parameter identifiers begin.

3.5.4 Component Map Data Dependencies

This component contains data dependencies that need their IDs mapped to valid data product IDs at initialization. Each data dependency also needs a stale limit provided in microseconds. If the timestamp of the data dependency is older than the reference time (usually the current time) minus the stale limit then the data dependency is considered too old to safely use. The `map_Data_Dependencies` procedure must be called with the following parameters:

Table 4: Rw Motor Torque Map Data Dependencies Parameters

Name	Type
`Control_Torque_Id`	`Data_Product_Types.Data_Product_Id`
`Control_Torque_Stale_Limit`	`Ada.Real_Time.Time_Span`
`Wheel_Speeds_Id`	`Data_Product_Types.Data_Product_Id`
`Wheel_Speeds_Stale_Limit`	`Ada.Real_Time.Time_Span`

Parameter Descriptions:

- **`Control_Torque_Id`** - The external data product ID used to fetch the value for the 'Control_Torque' data dependency.
- **`Control_Torque_Stale_Limit`** - The stale limit used for the 'Control_Torque' data dependency. If the fetched data product is older than the given Time_Span then it is deemed stale and unfit for use by the component.
- **`Wheel_Speeds_Id`** - The external data product ID used to fetch the value for the 'Wheel_Speeds' data dependency.
- **`Wheel_Speeds_Stale_Limit`** - The stale limit used for the 'Wheel_Speeds' data dependency. If the fetched data product is older than the given Time_Span then it is deemed stale and unfit for use by the component.

3.5.5 Component Implementation Initialization

The calling of this implementation class initialization procedure is mandatory. Initializes the reaction wheel motor torque algorithm with the default parameter values. The `init` subprogram takes no parameters.

3.6 Commands

The Rw Motor Torque component has no commands.

3.7 Parameters

Parameters for the Rw Motor Torque component.

Table 5: Rw Motor Torque Parameters

Local ID	Parameter Name	Type	Default Value
0x0000 (0)	Control_Axes	Rw_Motor_Torque_Control_Axes.T	(Torque_X=>True, Torque_Y=>True, Torque_Z=>True)
0x0001 (1)	Rw_Spin_Axes	Packed_F32x3_X4.T	[[1.0,0.0,0.0],[0.0,1.0,0.0],[0.0,0.0,1.0],[0.57735027, 0.57735027,0.57735027]]
0x0002 (2)	Wheel_Availability	Wheel_Availability_X4.T	[others=>Mrp_Feedback_Enums.Wheel_Availability.Available]
0x0003 (3)	Proportional_Gain_Kp	Packed_F32.T	(Value=>0.0)
0x0004 (4)	Desired_Wheel_Speeds	Rw_Speeds_Input.T	(Wheel_Speeds=>[others=>0.0])

Parameter Descriptions:

- **`Control_Axes`** - [-] Body axes the wheels are asked to control. At least one must be selected, and every selected axis must be reachable by the available wheels.
- **`Rw_Spin_Axes`** - [-] Reaction wheel spin axes in body frame components, one unit vector per wheel slot. Each must be finite and within 1e-3 of unit length.
- **`Wheel_Availability`** - [-] Which reaction wheels take part in the mapping. An unavailable wheel is commanded zero torque. Every slot is configured, so a slot without a wheel is marked unavailable.
- **`Proportional_Gain_Kp`** - [-] Null space feedback gain on the wheel speed error. Must be finite and not negative. Zero disables the null space term.
- **`Desired_Wheel_Speeds`** - [rad/s] Desired speed of each reaction wheel. The null space term steers the wheels toward these speeds when its gain is not zero. Each must be finite.

3.8 Events

The Rw Motor Torque component has no events.

3.9 Data Products

The Rw Motor Torque component has no data products.

3.10 Data Dependencies

Data dependencies for the Rw Motor Torque component.

Table 6: Rw Motor Torque Data Dependencies

Data Dependency Name	Type
`Control_Torque`	`Cmd_Torque_Body.T`
`Wheel_Speeds`	`Rwa_Speeds.T`

Data Dependency Descriptions:

- **`Control_Torque`** - [Nm] Commanded control torque on the spacecraft in body frame components.
- **`Wheel_Speeds`** - [rad/s] Current speed of each reaction wheel.

3.11 Packets

The Rw Motor Torque component has no packets.

3.12 Faults

The Rw Motor Torque component has no faults.

4 Unit Tests

The following section describes the unit test suites written to test the component.

4.1 *Rw_Motor_Torque_Tests* Test Suite

This is a unit test suite for the Rw Motor Torque component

Test Descriptions:

- **`Test`** - Run the algorithm on the control axis, availability, null space gain, and desired wheel speed cases of the Python reference test to ensure the Ada to C to C++ integration is sound.
- **`Test_Invalid_Parameter`** - Ensure a staged configuration the algorithm would reject is refused at validation.
- **`Test_Invalid_Data_Dependency`** - Ensure a data dependency with the wrong identifier is treated as a wiring defect and fails the tick's assertion.

5 Appendix

5.1 Preamble

This component contains no preamble code.

5.2 Packed Types

The following section outlines any complex data types used in the component in alphabetical order. This includes packed records and packed arrays that might be used as connector types, command arguments, event parameters, etc..

Cmd_Torque_Body.T:

Commanded control torque in body frame components, as produced by the MRP control laws.

Table 7: Cmd_Torque_Body Packed Record : 96 bits

Name	Type	Range	Size (Bits)	Start Bit	End Bit
Torque_Request_Body	Packed_F32x3.T	-	96	0	95

Field Descriptions:

- **Torque_Request_Body** - [Nm] Control torque requested

Data_Product.T:

Generic data product packet for holding arbitrary data types

Table 8: Data_Product Packed Record : 1096 bits *(maximum)*

Name	Type	Range	Size (Bits)	Start Bit	End Bit	Variable Length
Header	Data_Product_Header.T	-	72	0	71	-
Buffer	Data_Product_Types.Data_Product_Buffer_Type	-	1024	72	1095	Header.Buffer_Length

Field Descriptions:

- **Header** - The data product header
- **Buffer** - A buffer that contains the data product type

Data_Product_Fetch.T:

A packed record which holds information for a data product request.

Table 9: Data_Product_Fetch Packed Record : 16 bits

Name	Type	Range	Size (Bits)	Start Bit	End Bit
Id	Data_Product_Types.Data_Product_Id	0 to 65535	16	0	15

Field Descriptions:

- **Id** - The data product identifier

Data_Product_Header.T:

Generic data_product packet for holding arbitrary data_product types

Table 10: Data_Product_Header Packed Record : 72 bits

Name	Type	Range	Size (Bits)	Start Bit	End Bit
Time	Sys_Time.T	-	48	0	47
Id	Data_Product_Types. Data_Product_Id	0 to 65535	16	48	63
Buffer_Length	Data_Product_ Types.Data_Product_ Buffer_Length_Type	0 to 128	8	64	71

Field Descriptions:

- **Time** - The timestamp for the data product item.
- **Id** - The data product identifier
- **Buffer_Length** - The number of bytes used in the data product buffer

Data_Product_Return.T:

This record holds data returned from a data product fetch request.

Table 11: Data_Product_Return Packed Record : 1104 bits *(maximum)*

Name	Type	Range	Size (Bits)	Start Bit	End Bit	Variable Length
The_ Status	Data_ Product_ Enums. Fetch_ Status.E	0 => Success 1 => Not_Available 2 => Id_Out_Of_Range	8	0	7	-
The_ Data_ Product	Data_ Product. T	-	1096	8	1103	-

Field Descriptions:

- **The_Status** - A status relating whether or not the data product fetch was successful or not.
- **The_Data_Product** - The data product item returned.

Packed_F32.T:

Single component record for holding packed 32-bit floating point number.

Table 12: Packed_F32 Packed Record : 32 bits

Name	Type	Range	Size (Bits)	Start Bit	End Bit
Value	Short_Float	-3.40282e+38 to 3.40282e+38	32	0	31

Field Descriptions:

- **Value** - The 32-bit floating point number.

Packed_F32x3.T:

Packed 3 element vector of 32-bit floats.

Table 13: Packed_F32x3 Packed Array : 96 bits

Type	Range	Element Size (Bits)	Length	Total Size (Bits)
Short_Float	-3.40282e+38 to 3.40282e+38	32	3	96

Packed_F32x3_X4.T:

Array of 4 Packed_F32x3 elements, one three-vector per reaction wheel.

Table 14: Packed_F32x3_X4 Packed Array : 384 bits

Type	Range	Element Size (Bits)	Length	Total Size (Bits)
Packed_F32x3.T	-	96	4	384

Packed_F32x4.T:

Table 15: Packed_F32x4 Packed Array : 128 bits

Type	Range	Element Size (Bits)	Length	Total Size (Bits)
Short_Float	-3.40282e+38 to 3.40282e+38	32	4	128

Parameter.T:

Generic parameter packet for holding a generic parameter

Table 16: Parameter Packed Record : 792 bits *(maximum)*

Name	Type	Range	Size (Bits)	Start Bit	End Bit	Variable Length
Header	Parameter_ Header.T	-	24	0	23	-
Buffer	Parameter_ Types. Parameter_ Buffer_Type	-	768	24	791	Header.Buffer_ Length

Field Descriptions:

- **Header** - The parameter header
- **Buffer** - A buffer that contains the parameter type

Parameter_Header.T:

Generic parameter header for holding arbitrary parameters

Table 17: Parameter_Header Packed Record : 24 bits

Name	Type	Range	Size (Bits)	Start Bit	End Bit

Id	Parameter_Types. Parameter_Id	0 to 65535	16	0	15
Buffer_Length	Parameter_Types. Parameter_Buffer_ Length_Type	0 to 96	8	16	23

Field Descriptions:

- **Id** - The parameter identifier
- **Buffer_Length** - The number of bytes used in the parameter type buffer

Parameter_Update.T:

A record intended to be used as a provide/modify connector type for updating/fetching parameters.

Table 18: Parameter_Update Packed Record : 824 bits *(maximum)*

Name	Type	Range	Size (Bits)	Start Bit	End Bit	Variable Length
Table_Id	Parameter_ Types. Parameter_ Table_Id	0 to 65535	16	0	15	-
Operation	Parameter_ Enums. Parameter_ Operation_ Type.E	0 => Stage 1 => Update 2 => Fetch 3 => Validate 4 => Discard	8	16	23	-
Status	Parameter_ Enums. Parameter_ Update_ Status.E	0 => Success 1 => Id_Error 2 => Validation_Error 3 => Length_Error	8	24	31	-
Param	Parameter. T	-	792	32	823	-

Field Descriptions:

- **Table_Id** - The ID for the table that contains this parameter
- **Operation** - The parameter operation to perform.
- **Status** - The parameter return status.
- **Param** - The parameter that has been updated or fetched.

Rw_Motor_Torque_Control_Axes.T:

Body axes the reaction wheels are asked to control, x then y then z. A true entry selects the axis. Maps to RwMotorTorqueControlAxes_c in the rwMotorTorque C shim, which takes it by pointer.

Table 19: Rw_Motor_Torque_Control_Axes Packed Record : 24 bits

Name	Type	Range	Size (Bits)	Start Bit	End Bit
Torque_X	Boolean	0 => False 1 => True	8	0	7
Torque_Y	Boolean	0 => False 1 => True	8	8	15
Torque_Z	Boolean	0 => False 1 => True	8	16	23

Field Descriptions:

- **Torque_X** - [-] Control the body x torque axis
- **Torque_Y** - [-] Control the body y torque axis
- **Torque_Z** - [-] Control the body z torque axis

Rw_Speeds_Input.T:

Table 20: Rw_Speeds_Input Packed Record : 128 bits

Name	Type	Range	Size (Bits)	Start Bit	End Bit
Wheel_Speeds	Packed_F32x4.T	-	128	0	127

Field Descriptions:

- **Wheel_Speeds** - [rad/s] Reaction wheel speeds

Rwa_Speeds.T:

Reaction Wheel Assembly (RWA) wheel speeds in rad/s.

Table 21: Rwa_Speeds Packed Record : 128 bits

Name	Type	Range	Size (Bits)	Start Bit	End Bit
Rwa_1	Short_Float	-3.40282e+38 to 3.40282e+38	32	0	31
Rwa_2	Short_Float	-3.40282e+38 to 3.40282e+38	32	32	63
Rwa_3	Short_Float	-3.40282e+38 to 3.40282e+38	32	64	95
Rwa_4	Short_Float	-3.40282e+38 to 3.40282e+38	32	96	127

Field Descriptions:

- **Rwa_1** - Wheel speed of RWA 1 in rad/s.
- **Rwa_2** - Wheel speed of RWA 2 in rad/s.
- **Rwa_3** - Wheel speed of RWA 3 in rad/s.
- **Rwa_4** - Wheel speed of RWA 4 in rad/s.

Rwa_Torques.T:

Reaction Wheel Assembly (RWA) torques.

Table 22: Rwa_Torques Packed Record : 128 bits

Name	Type	Range	Size (Bits)	Start Bit	End Bit
Rwa_1	Short_Float	-3.40282e+38 to 3.40282e+38	32	0	31
Rwa_2	Short_Float	-3.40282e+38 to 3.40282e+38	32	32	63
Rwa_3	Short_Float	-3.40282e+38 to 3.40282e+38	32	64	95
Rwa_4	Short_Float	-3.40282e+38 to 3.40282e+38	32	96	127

Field Descriptions:

- **Rwa_1** - Torque to apply to RWA 1 in NM.
- **Rwa_2** - Torque to apply to RWA 2 in NM.
- **Rwa_3** - Torque to apply to RWA 3 in NM.
- **Rwa_4** - Torque to apply to RWA 4 in NM.

Sys_Time.T:

A record which holds a time stamp using EMA VTC format including seconds and subseconds since epoch (1-5-1980 to 1-6-1980 midnight).

Table 23: Sys_Time Packed Record : 48 bits

Name	Type	Range	Size (Bits)	Start Bit	End Bit
Seconds	Interfaces. Unsigned_32	0 to 4294967295	32	0	31
Subseconds	Interfaces. Unsigned_16	0 to 65535	16	32	47

Field Descriptions:

- **Seconds** - The number of seconds elapsed since epoch.
- **Subseconds** - The number of 1/(2^16) sub-seconds.

Tick.T:

The tick datatype used for periodic scheduling. Included in this type is the Time associated with a tick and a count.

Table 24: Tick Packed Record : 80 bits

Name	Type	Range	Size (Bits)	Start Bit	End Bit
Time	Sys_Time.T	-	48	0	47
Count	Interfaces. Unsigned_32	0 to 4294967295	32	48	79

Field Descriptions:

- **Time** - The timestamp associated with the tick.
- **Count** - The cycle number of the tick.

Wheel_Availability_X4.T:

Availability of each of the four reaction wheels, one byte per slot as the C boundary carries it. Values outside the enumeration fail type validation during parameter staging.

Table 25: Wheel_Availability_X4 Packed Array : 32 bits

Type	Range	Element Size (Bits)	Length	Total Size (Bits)
Mrp_Feedback_Enums.Wheel_Availability.E	0 => Available 1 => Unavailable	8	4	32

5.3 Enumerations

The following section outlines any enumerations used in the component.

Data_Product_Enums.Fetch_Status.E:

This status denotes whether a data product fetch was successful.

Table 26: Fetch_Status Literals:

Name	Value	Description
Success	0	The data product was returned successfully.
Not_Available	1	No data product is yet available for the provided id.
Id_Out_Of_Range	2	The data product id was out of range.

Mrp_Feedback_Enums.Wheel_Availability.E:

Whether a reaction wheel contributes to the wheel momentum term. Values match the C DeviceAvailability_c enumeration.

Table 27: Wheel_Availability Literals:

Name	Value	Description
Available	0	The wheel is running and its momentum is included.
Unavailable	1	The wheel is excluded from the momentum term.

Parameter_Enums.Parameter_Operation_Type.E:

This enumeration lists the different parameter operations that can be performed.

Table 28: Parameter_Operation_Type Literals:

Name	Value	Description
Stage	0	Stage the parameter.
Update	1	All parameters are staged, it is ok to update all parameters now.
Fetch	2	Fetch the parameter.
Validate	3	Validate all the parameters.
Discard	4	Discard any parameters staged by this table that have not been updated.

Parameter_Enums.Parameter_Update_Status.E:

This status enumeration provides information on the success/failure of a parameter operation.

Table 29: Parameter_Update_Status Literals:

Name	Value	Description
Success	0	Parameter was successfully staged.
Id_Error	1	Parameter id was not valid.
Validation_Error	2	Parameter values were not successfully validated.
Length_Error	3	Parameter length was not correct.